

# General information

Title : Kitchen LED project overview

Eng : Michael

Master description revision : v2 (09/14/2025)

## Change sheet

| When?      | What?                                                                                 |
|------------|---------------------------------------------------------------------------------------|
| 09/14/2025 | Created the document, filled in basic info. Concept description and initial analysis. |
| 09/14/2025 | Created server requirements, frozen the config into v1.                               |
|            |                                                                                       |

## References

Master description file

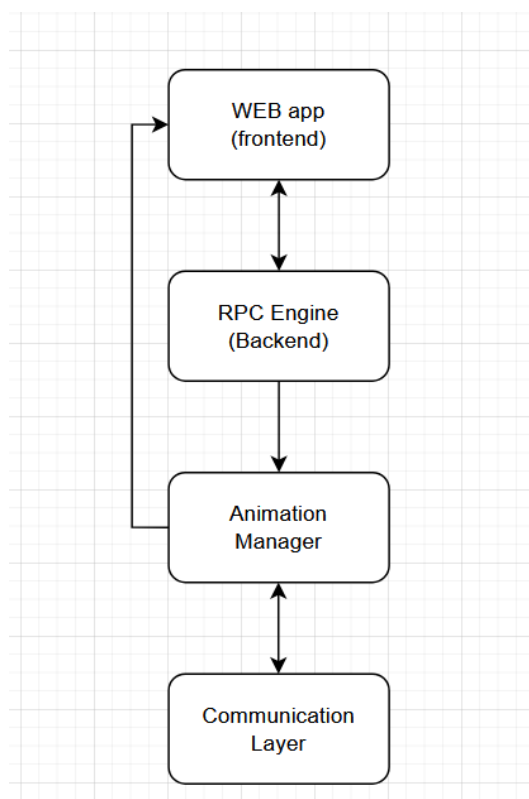
[https://drive.google.com/file/d/1w6MIWb4a44Vop6j\\_Z\\_MQZbftWPQRvtq-/view?usp=drive\\_link](https://drive.google.com/file/d/1w6MIWb4a44Vop6j_Z_MQZbftWPQRvtq-/view?usp=drive_link)

|                                |          |
|--------------------------------|----------|
| <b>General information</b>     | <b>1</b> |
| Change sheet                   | 1        |
| References                     | 1        |
| <b>Brief</b>                   | <b>3</b> |
| Block schematic - initial idea | 3        |
| Initial analysis               | 4        |
| SBC / SoC linux host           | 4        |
| Web app                        | 4        |
| RPC engine                     | 4        |
| Animation manager              | 4        |
| Communication layer            | 4        |
| <b>Server requirements</b>     | <b>5</b> |

# Brief

This document describes the UI host (server) part of the project. As per the frozen config, this block should be implemented as an independent part, an application hosted on SBC (single board computer) sourcing from the system requirements.

## Block schematic - initial idea



- WEB app - allows setting working parameters and read basic device status like connection status or if operation execution succeeded
- RPC engine - callback dispatcher mechanism - verifies user input and passes into the lower layers
- Animation manager handles request and manages the animation by building packets and passing them to the communication layer
- Communication layer ensures that link to the device is available and sends requests to change the strip colour/behavior

**NOTE:** for the purposes of the project, it's enough to assume that all LEDs will emit the same colour at the same time.

## Initial analysis

### SBC / SoC linux host

Currently, the market offers a variety of solutions that can be used for embedded systems development:

- SBC - Odroid, Raspberry Pi, <any-fruit> Pi variants, Nvidia Jetson
- SoC - e.g., Xilinx Zynq series chips

**Final decision:** SoC-based solutions are well-suited for serial production and tailored, highly customized systems. However, for the purposes of this project, it is sufficient to use an SBC. The products listed above offer strong community support and are essentially plug-and-play.

It is safe to assume that a Raspberry Pi is more than enough for this application - it is inexpensive, well-tested, and widely supported.

### Web app

Basic application that allows setting led strip parameters, creating animation sequences and reading system status.

### RPC engine

Google RPC-based application that handles requests coming from the web application - verification and passing down the layers.

### Animation manager

Application that handles currently selected animations - strip colour, effects like fade, strip supply. Runs periodically, builds packets and passes to the communication layer - gives feedback to the UI for the user.

### Communication layer

Manages physical connection with the driver. It ensures reliable packet transmission and provides feedback to the Animation Manager.

# Server requirements

| master<br>REQ_ID | REQ_ID | Description                                                                                                               |
|------------------|--------|---------------------------------------------------------------------------------------------------------------------------|
| SYS_2<br>SYS_5   | COM_1  | Communication layer should handle link status and requests from Animation Manager; report back to the Animation manager.  |
| SYS_2<br>SYS_5   | COM_2  | Communication layer should be able to send and receive packets over UART.                                                 |
| SYS_2<br>SYS_5   | COM_3  | Communication layer should use unified communication protocol for data exchange.                                          |
| SYS_6            | CFG_1  | Before the application init, IP should be configured to static or a domain name should be available in the local network. |
| SYS_7<br>SYS_8   | UI_1   | UI should show the device and strip basic status.                                                                         |
| SYS_7<br>SYS_8   | UI_2   | UI should allow changing basic working parameters of the driver.                                                          |
| SYS_7<br>SYS_8   | UI_3   | UI should allow creating animations (changing colours and transition effects).                                            |
| SYS_7<br>SYS_8   | RPC_1  | RPC handler should verify requests from the UI and pass it down to the animation manager.                                 |
| SYS_7<br>SYS_8   | ANIM_1 | Animation manager should periodically change strip status.                                                                |
| SYS_7<br>SYS_8   | ANIM_2 | Default animation is strip turned off.                                                                                    |